Yesterday 10:55 PM

Lab_4_VibeCoding...
PDF

5H_VibeCodingGit...
Spreadsheet

mine is PES1UG24CS450, so tell me properly step by step what to do without missing any step

Worked for 3m 34s

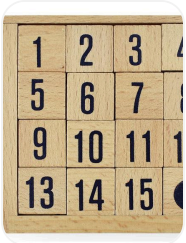
Analyzed

Analyzed

Analyzed

Analyzed

Your SRN **PES1UG24CS450** is assigned **Lab 4 — Scenario 22: Sliding Puzzle**.



Your assigned repository

22_sliding_puzzle

A Python terminal-based sliding puzzle.

Open your assigned GitHub repository ↗

You must fix the existing code, implement the four tasks in its README, test your changes, and push the finished code to your own GitHub repository. Do not create a pull request to SETAPESU26.

I checked your assigned repository and its README. Your project contains `main.py`, `game.py`, `puzzle.py` and `requirements.txt`. It requires Python 3.9 or later and uses only the Python standard library, so you do not need to install extra Python packages. SETAPESU26/... +1

Follow these steps **in this order**.

Step 1: Prepare your computer

You will need Git, Python and a code editor such as VS Code.

1.1 Check Python

Open **Windows PowerShell** or the VS Code terminal and run:

powershell

```
py --version
```

You should see Python 3.9 or a newer version.

If `py` is not recognised, try:

powershell

```
python --version
```

If Python is not installed, download it from python.org ↗. On Windows, select the option to add Python to PATH during installation if it is offered.

1.2 Check Git

Run:

powershell

```
git --version
```

If Git is not installed, get it from git-scm.com ↗.

Do not start modifying your puzzle code yet. First, secure your own copy of the assigned repository and record the original behaviour.

Step 2: Make a copy under your personal GitHub account

This step is important because the original repository belongs to `SETAPESU26`, not your personal account.

1. Open the assigned repository ↗.
2. Sign in to your own GitHub account.
3. Click **Fork** near the top-right of the repository.
4. Select your own GitHub account as the destination.
5. Keep the repository name as `22_sliding_puzzle`, unless GitHub requires a different name.
6. Click **Create fork**.

You should now have your own repository at an address similar to:

`https://github.com/YOUR_USERNAME/22_sliding_puzzle`

Replace `YOUR_USERNAME` with your real GitHub username.

Important: You will edit and push to your personal copy, not directly to `SETAPESU26/22_sliding_puzzle` . Do not raise a pull request.

If you already have a fork of this repository, open that existing fork instead of creating another one.

Step 3: Download your personal copy to your computer

In your personal fork on GitHub:

1. Click the green **Code** button.
2. Select **HTTPS**.
3. Copy the repository URL.
4. Open PowerShell.
5. Run the following commands, using the URL you copied.

```powershell
cd $HOME\Desktop
git clone https://github.com/YOUR_USERNAME/22_sliding_puzzle.git
cd 22_sliding_puzzle
```

Replace the URL with your actual fork URL.

Now verify where Git will push your changes:

```powershell
git remote -v
```

The output should show your **personal GitHub username** in the repository URL for both fetching and pushing.

If the URL still points to `SETAPESU26/22_sliding_puzzle` , stop and correct your setup before pushing.

Step 3.1: Open the folder in VS Code

While you are inside the project folder, run:

```powershell
code .
```

If that command does not work, open VS Code manually, select **File** → **Open Folder**, and choose the `22_sliding_puzzle` folder on your Desktop.

You should see these files:

```
22_sliding_puzzle/
├── README.md
├── requirements.txt
├── main.py
├── game.py
└── puzzle.py
```

These are the original project files identified in the repository README. SETAPESU26/... +3

Step 4: Run the original, broken code

You must do this **before changing any code**, because the assignment requires a 10-second video of the original game.

In the VS Code terminal, make sure you are in the project folder, then run:

```powershell
py main.py
```

If that does not work but the `python` command does, use:

```powershell
python main.py
```

The game should open in the terminal and display a numbered puzzle. Its original controls are W/A/S/D for moves and Q to quit. The initial board is generated by randomly shuffling the tiles. This is the defect you are investigating: a random permutation can produce a board that cannot be solved using legal moves. SETAPESU26/... +2

Try restarting the program a few times so you can observe its initial boards. Exit the game by entering:

```
q
```

Step 4.1: Record your BEFORE video

Record this now, while the code is still unchanged.

For Windows, you can use **Snipping Tool's screen-recording feature** or Xbox Game Bar.

1. Open Snipping Tool and select its recording mode, or open Xbox Game Bar with `Win + G`.
2. Select the terminal window or screen area containing your game.
3. Start recording.
4. Run the original game and keep its initial board visible for at least 10 seconds.
5. Stop the recording.
6. Save the video as `before.mp4`.

Optional: Reproduce the original defect more reliably

Because an impossible sliding-puzzle board looks much like an ordinary board, you can run the original code with a fixed random seed to demonstrate the defect without editing any files.

Run this command from your project directory:

```powershell
```

```
py -c "import random; random.seed(2); from game import SlidingPuzzle; SlidingPuzzle().run()"
```

With a compatible Python runtime, the original code should display this board:

Plain text

```
13 12  8 14
 7  1  9  4
10  5 11  3
 6 15     2
```

This is an impossible-parity starting state for the 4×4 puzzle. Keep that board visible in your recording. If it does not appear, use the normal launch command and capture the original behaviour instead.

Do not edit the Python files before you finish this recording.

Step 5: Read the README and identify the four tasks

Before asking an AI coding tool to make changes, open `README.md` in VS Code.

Your assignment requires these four tasks to be implemented. Use the README as the specification; do not replace the game with a different project. SETAPESU26/22_...

Task 1 — Guarantee solvable starting states

Every new puzzle must be solvable in principle. Prefer starting with the solved board and scrambling it by making legal moves, rather than arbitrarily shuffling the tiles.

Task 2 — Complete the puzzle lifecycle

Detect when the puzzle is solved, end the game cleanly and prevent further moves after completion.

Task 3 — Add size, timer and move modes

Support 3×3, 4×4 and 5×5 puzzles. Track elapsed time and move counts consistently, without accidentally resetting session values when recreating a board.

Task 4 — Give correct valid-action feedback

Report a successful move only when a tile actually moves. Invalid moves must not increase the move count or display success feedback.

Step 6: Use ChatGPT, Claude or another Vibe Coding tool

The goal is to repair the **existing code**. Work on one task at a time and test it before moving on to the next. Keep the entire conversation, because it is one of your deliverables.

You can use the prompts below in a single conversation. If your AI tool cannot read your local files, attach or paste `README.md`, `main.py`, `game.py`, `puzzle.py` and `requirements.txt`.

6.1 First prompt — Understand the existing code

Send this first. It is for analysis only, so do not ask for a rewrite yet.

PROMPT 1 — INSPECT THE PROJECT

I am completing Lab 4, Scenario 22 — Sliding Puzzle. Read the README and inspect the existing main.py, game.py, puzzle.py and requirements.txt.

Explain the current program flow, how the puzzle is initialised, how moves are handled, and why random tile shuffling can create an unsolvable starting board.

Identify the exact files and functions that need changes for each of the four README tasks. Do not rewrite the whole project or change any code yet. Keep the solution modular, understandable and within the README requirements.

Copy prompt

Read the answer and make sure you understand the proposed changes before applying them.

6.2 Second prompt — Fix Task 1

PROMPT 2 — SOLVABILITY

Implement only Task 1 from the README: guarantee solvable starting states.

Prefer generating the solved arrangement and scrambling it through a sequence of legal blank moves. Preserve the existing Puzzle class and modular project structure. Do not add unnecessary

third-party packages, persistent storage, or unrelated features.

Show the exact changes made. Explain why every generated board is reachable from the solved arrangement, and give me a practical test for checking many freshly generated boards. Do not implement Tasks 2–4 yet.

Copy prompt

Apply the suggested changes to your local files and run the program again:

```powershell
py main.py
```

Check that the game launches and the board displays correctly. If it fails, resolve the error before proceeding.

6.3 Third prompt — Fix Task 2

PROMPT 3 — GAME COMPLETION

Implement only Task 2 from the README.

Detect the exact solved arrangement, display a clear completion message, end the game cleanly, and prevent additional commands from changing the state after the puzzle is solved. Preserve the existing code structure.

Include a test for an already-solved board and a test for a board that becomes solved after a legal move. Explain how I can verify that the game exits correctly. Do not implement Tasks 3–4 yet.

Copy prompt

Apply the changes, run the game and verify that completion is detected correctly.

6.4 Fourth prompt — Fix Task 3

PROMPT 4 — BOARD SIZES, TIMER AND MOVES

Implement only Task 3 from the README.

Add support for 3x3, 4x4 and 5x5 boards. Ensure the board dimensions, tile arrangement, blank position and solved-state detection work for every size.

Track valid moves and elapsed time consistently. Invalid moves must not increase the move count. Do not accidentally reset session values when recreating a board; keep board state and session statistics appropriately separated.

Preserve the existing modular files, use no unnecessary dependencies or persistence, and show me how to test all three board sizes. Do not implement Task 4 yet.

Copy prompt

Apply the changes and verify that all three sizes work before continuing.

6.5 Fifth prompt — Fix Task 4

PROMPT 5 — VALID AND INVALID MOVES

Implement only Task 4 from the README.

Check all movement inputs carefully. Show successful-move feedback only when a tile is legally moved into the blank. For impossible edge moves, invalid inputs and other rejected actions, do not increase the move count or report success.

Keep the game's direction labels consistent with the actual movements. Preserve the existing modular design and do not introduce unnecessary dependencies, persistence, or unrelated features.

Give me tests for valid moves, impossible moves at board edges, invalid commands, unchanged move counts after rejected moves, and quitting the game. Show the exact changes and explain them.

Copy prompt

Apply the final changes and rerun the entire project:

```powershell
```

```
py main.py
```

Important: Do not stop just because the AI says the code is fixed. You must run it yourself and verify the actual behaviour.

Step 7: Test everything before submitting

The README explicitly calls for testing fresh boards, every board size, solving a small board, impossible moves, move counts, timer behaviour, invalid commands and quitting. SETAPESU26/22_...

Use this checklist in order.

Testing checklist

0 of 18 complete

Starting states

- [ ] Start the game repeatedly and generate at least 10 fresh boards.
- [ ] Verify that each generated board is solvable in principle, including its parity.
- [ ] Check that the blank tile and all numbered tiles appear exactly once.

All board sizes

- [ ] Start a 3×3 puzzle and check its dimensions.
- [ ] Start a 4×4 puzzle and check its dimensions.
- [ ] Start a 5×5 puzzle and check its dimensions.
- [ ] Check that solved-state detection works for each size.

Moves and feedback

- [ ] Make a valid move and confirm the move count increases by exactly one.
- [ ] Attempt an impossible move at an edge and confirm the board and move count do not change.
- [ ] Enter an invalid command, such as x, and check that it is rejected.
- [ ] Confirm that feedback and the movement directions are consistent.

Timer and completion

- [ ] Confirm that elapsed time increases while playing.
- [ ] Confirm that recreating a board does not unexpectedly reset session values.
- [ ] Solve a small puzzle and verify that the game displays a completion message and ends correctly.

- [ ] Verify that no further moves can change the state after completion.

Exit and final run

- [ ] Enter q and confirm the game quits cleanly.
- [ ] Restart the program and check for errors.
- [ ] Review the changed files and ensure no unrelated project or persistence features were added.

Reset checklist

If you need an easy test of the solved state

For testing the puzzle logic independently, you can use the current `Puzzle` class to create a board that is one move away from solved:

powershell

```
py -c "from puzzle import Puzzle; p=Puzzle(3); p.board=[[1,2,3],[4,5,6],[7,0,8]]; print('Before:',p.solved());
```

This should print `Before: False` and `After: True` if your implementation retains the same class interface and direction mapping. It is a small logic test, not a substitute for checking the complete game's win-and-exit behaviour.

If any test fails, return to your Vibe Coding conversation, explain the exact failure and ask for a targeted fix. Do not ask the tool to regenerate your entire project.

Step 8: Record your AFTER video

Once all four tasks pass testing, record the changed game.

1. Open the finished application.
2. Start recording with the same screen recorder you used for the first video.
3. Show the size-selection options for 3×3, 4×4 and 5×5.
4. Demonstrate gameplay and a valid move.
5. If you can fit it in the recording, show an invalid move being rejected without increasing the move count.
6. Show the timer or move counter changing appropriately.
7. Stop recording after at least 10 seconds and save it as `after.mp4`.

Try to make the difference between the original and the fixed program clear. You can record a little longer than 10 seconds if necessary to show the new features properly.

Keep both `before.mp4` and `after.mp4`. Do not replace the original video with the after video.

Step 9: Review and push the corrected code to your personal fork

Return to your VS Code terminal. Check which files you changed:

```powershell
git status
```

Review the actual code changes:

```powershell
git diff
```

Check for whitespace errors:

```powershell
git diff --check
```

Fix any problems, rerun your tests and make sure that you are happy with the finished code.

Now stage the changes and commit them:

```powershell
git add .
git commit -m "Complete Lab 4 sliding puzzle tasks"
```

Push the finished code to your own GitHub repository:

```powershell
git push origin main
```

If your local branch is named something other than `main`, use the branch name shown by:

```powershell
git branch --show-current
```

Open your personal GitHub fork in your browser and refresh it. Confirm that the corrected `main.py`, `game.py`, `puzzle.py` and any legitimate additional test files are visible.

Do not open or submit a pull request to the original `SETAPESU26` repository. Your assignment requires pushing to your own repository, not to the original. Lab_4_VibeCoding_Student_handout.pdf

Step 10: Prepare your Lab-4 submission folder

Your student handout says to upload the **before video, after video, updated code and exported chat history** to your Lab repository under a `Lab-4` folder. Lab_4_VibeCoding_Student_handout.pdf

Keep the corrected game on your personal fork from Step 9. The separate Lab submission repository is for the deliverables listed below.

10.1 Create this folder structure

- **Your Lab repository**
  - **Lab-4/**
    - before.mp4 — **Required**
    - after.mp4 — **Required**
    - **Updated-Code/**
      - main.py
      - game.py
      - puzzle.py
      - requirements.txt

README.md

Any additional test files you created

chat-history.pdf

chat-link.txt (recommended)

Do not copy the nested `.git` directory into `Updated-Code`.

10.2 Export your complete AI conversation

Your README asks for the complete LLM chat history, and your student handout specifically asks for an exported document or PDF. The safest approach is to provide both the PDF and the conversation link.

SETAPESU26/22_...

1. Keep all your diagnosis, implementation prompts, AI responses, fixes and testing questions in one conversation.
2. Use your AI tool's sharing or export option to obtain the conversation link, if available.
3. Copy the full conversation, including the prompts and responses, into Word if necessary.
4. Save or export it as `chat-history.pdf`.
5. Put the share URL in `chat-link.txt`.

Make sure the PDF contains the actual conversation, not just a summary. Check that the exported PDF opens and that the text is readable.

10.3 Push the Lab-4 folder

Open your separate course Lab repository. Create the `Lab-4` folder, place the files inside it, and commit and push the additions. If that repository is already cloned on your computer, use its own terminal and Git commands rather than running them from the sliding puzzle repository.

Once uploaded, open the Lab repository on GitHub and verify that every deliverable is visible.

Final submission checklist

Ready to submit? 0/10

- [ ] Personal fork created for 22_sliding_puzzle.
- [ ] Original broken-game video saved as before.mp4.
- [ ] All four README tasks implemented in the existing project.
- [ ] Every board size, valid move, invalid move, timer and completion behaviour tested.

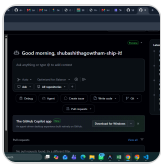
- [ ] Corrected code committed and pushed to my personal fork.
- [ ] After video saved as after.mp4.
- [ ] Updated source files included under Lab-4/Updated-Code/.
- [ ] Complete AI chat exported to chat-history.pdf.
- [ ] AI conversation link saved, if available.
- [ ] Lab-4 folder uploaded to my course Lab repository and verified.

One last detail: The `BugFree Code - Git Repo Link` column for your SRN was blank in the provided spreadsheet. After pushing your changes, keep your personal fork's URL ready in case your instructor expects you to submit or enter it there.

The order to remember is: **Fork → Clone → Run original code → Record before video → Fix Tasks 1–4 one at a time → Test → Record after video → Push corrected code → Export chat → Upload all deliverables under** `Lab-4` **.**

‹/›

Today 5:58 AM



Perfect, you're on your GitHub homepage and you're already logged in. ✅ We'll do this **one step at a time**.

Step 1 — Open your assigned repository

Click this link:

Open your Sliding Puzzle repository — Scenario 22 ↗

Step 2 — Fork the repository

Once it opens:

1. Look at the **top-right corner** of the repository page.
2. Click **Fork**.
3. Under **Owner**, select your personal GitHub account, `shubashithagowtham-ship-it` .
4. Keep the repository name as `22_sliding_puzzle` .

5. Click **Create fork**.

This makes your own copy of the assignment so you can modify and submit your work without changing the original repository.

Stop after Step 2 and send me a screenshot of the page you see. I'll guide you through the next step based on your actual screen.

done

Good! ✅ Now we'll download your fork to your computer so you can run and fix the Sliding Puzzle game.

Step 2 — Copy your repository link

1. Open your personal fork here: Your GitHub repositories ↗
2. Click **22_sliding_puzzle**.
3. On the repository page, click the green **Code** button.
4. Select **HTTPS**.
5. Click the copy icon next to the URL.

It should look similar to this:

```
https://github.com/shubashithagowtham-ship-it/22_sliding_puzzle.git
```

Don't worry if the URL looks slightly different.

Once you've copied the link, tell me "done." Then we'll clone it into your computer using VS Code, one command at a time.

done

Great! ✅ Now we'll clone your repository onto your Windows computer.

Step 3 — Open VS Code

Look at the taskbar at the bottom of your screen and click the **blue VS Code icon**.



Once VS Code opens:

1. Click **Terminal** in the top menu.
2. Click **New Terminal**.
3. A terminal panel will open at the bottom.

In the terminal, type this command and press **Enter**:

```powershell
cd $HOME\Desktop
```

This takes you to your Desktop, where we'll download the project.

Tell me when you've done this. We'll run the next command together.

done

Perfect! Now let's download your Sliding Puzzle project.

Step 4 — Clone your repository

In the VS Code terminal, copy and paste this command:

```powershell
git clone https://github.com/shubashithagowtham-ship-it/22_sliding_puzzle.git
```

Press **Enter** and wait for it to finish. You should see messages about downloading and receiving files.

If it asks you to authenticate or shows an error, don't worry — send me the exact message or a screenshot.

Once cloning finishes, tell me "done." Then we'll open the project folder and run the original game before making any changes.

done

Great! ✅ Your repository should now be downloaded to your Desktop.

Step 5 — Enter your project folder

Go to the VS Code terminal and type:

```powershell
```

```
cd 22_sliding_puzzle
```

Press **Enter**.

Now type:

```powershell
dir
```

Press **Enter** again.

You should see files such as `README.md`, `main.py`, `game.py`, `puzzle.py` and `requirements.txt`.

Send me a screenshot of the terminal output. I'll confirm that you're in the correct folder before we run the original game. Remember, we need to record the 10-second BEFORE video before changing any code.

yes

Okay bro, let's run your original game now. ❤️ Don't change any code yet!

Step 6 — Run the Sliding Puzzle

In your VS Code terminal, type:

```powershell
py main.py
```

Press **Enter**.

What should happen?

The Sliding Puzzle game should start in your terminal and display a board containing numbered tiles and an empty space.

- **W / A / S / D** — move the tiles.

- **Q** — quit the game.

If you see an error such as `py is not recognized` or `No module named ...`, send me a screenshot before trying anything else.

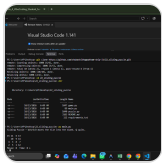
Step 7 — Record your BEFORE video

Once the game runs successfully:

1. Press `Win + G` to open Xbox Game Bar.
2. Start recording with `Win + Alt + R`.
3. Keep the original puzzle visible for at least **10 seconds**.
4. Stop recording using `Win + Alt + R`.
5. Find the recording in **Videos** → **Captures** and keep it as `before.mp4` (rename it if necessary).

This recording must be made before you modify the code.

First, run `py main.py` and tell me what appears on your screen.

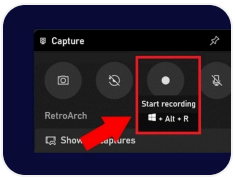


YESS, your game is running correctly! ✅ 🎉 You're in the right folder, and the original Sliding Puzzle is displayed with **Moves: 0** and **Time: 0 s**.

Don't change any code yet. First, we must save your BEFORE video for the assignment.

Step 7 — Record your BEFORE video

Follow these steps on the screen you just sent.



1. Start recording

Press these keys together:

Win + Alt + R

Look for a small recording timer or recording indicator on your screen.

2. Keep the game visible

Leave the puzzle running for at least **10 seconds**. Keep the board and its move count visible. You don't need to make any moves.

3. Stop recording

Press **Win** + **Alt** + **R** again.

4. Find your video

Open File Explorer and go to:

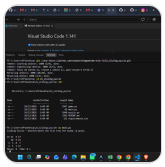
`Videos → Captures`

Rename the video to `before.mp4` if needed.

If the shortcut doesn't start recording, press **Win** + **G** to open Xbox Game Bar and look for the Capture widget.

Do this now and tell me when `before.mp4` is saved. Then we'll move on to fixing the code, one task at a time.

Today 11:46 AM



what should i do after this

You're in the correct place! ❤️ Your Sliding Puzzle is running. Now we'll move to the next step without missing anything.

Step 1 — Make sure your BEFORE video is saved

Your screenshot shows the original game still running, so **do not edit any code yet**.

Have you recorded at least 10 seconds and saved `before.mp4` ?

- If not, press **Win** + **Alt** + **R**, wait at least 10 seconds, then press **Win** + **Alt** + **R** again. Check `Videos → Captures` for the recording.
- If yes, continue to Step 2.

Step 2 — Stop the original game

Click inside the terminal at the bottom of VS Code and type:

Plain text

```
q
```

Press **Enter** if needed. This should quit the game.

Step 3 — Open your project properly in VS Code

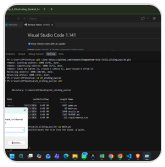
Your screenshot says "**No Folder Opened**" in the left Explorer panel. Let's fix that.

1. Click **File** in the top-left corner of VS Code.
2. Click **Open Folder**.
3. Navigate to:

   `C:\Users\HP\Desktop\22_sliding_puzzle`
4. Select the `22_sliding_puzzle` folder.
5. Click **Select Folder**.
6. If VS Code asks whether you trust the folder, select **Trust** because this is the repository you cloned.

You should now see `README.md`, `main.py`, `game.py`, `puzzle.py` and `requirements.txt` in the left Explorer panel.

Do these steps and send me a screenshot. Next, we'll start fixing the first bug using your Vibe Coding tool, while keeping your original project intact.



this comes when i click win+alt+R

Don't worry 😭 ❤️ Nothing is broken! That is the Windows **Run** dialog, not the screen recorder. You probably pressed `Win + R` instead of the recording shortcut.

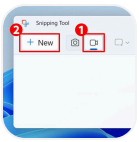
Let's close it and use a simpler method.

Step 1 — Close this popup

Click **Cancel** at the bottom of the small Run window.

Do not change anything in the terminal behind it. Your puzzle game should still be running.

Step 2 — Open Snipping Tool



Find the screen recorder

1. Press the **Windows key** on your keyboard.
2. Type `Snipping Tool`.
3. Click the Snipping Tool application in the search results.

Choose video recording

1. Look for the **Record** option or video-camera icon.
2. Click **Record**, then **New**.
3. Drag a rectangle around the VS Code terminal showing your puzzle.
4. Click **Start**.

Save your video

1. Leave the puzzle visible for at least 10 seconds.
2. Click **Stop**.
3. Click the save icon.
4. Save the recording on your Desktop as `before.mp4`.

Do this first: close the popup and open Snipping Tool. Send me a screenshot of Snipping Tool when it opens, and I'll point you to the exact recording button. We won't modify your code until the original video is saved.

Release Notes_ 1.1...
File

this video only?

Worked for 26s

YESSS, this video is enough for your BEFORE recording! ✅ ❤️

I checked the video you sent.

- **Duration:** Approximately 15 seconds, so it meets the 10-second requirement.
- **Original game:** Visible and running in the terminal.
- **Move count and timer:** Visible.
- **Code changes:** You haven't modified the code yet, which is exactly what we need.

Do this now

Step 1: Save the video somewhere easy to find, such as your Desktop, and rename it:

```
before.mp4
```

Step 2: Let's open your project properly in VS Code.

1. Click inside the terminal where the puzzle is running.
2. Type `q` and press Enter to exit the game.
3. Click **File** → **Open Folder** in VS Code.
4. Select this folder:

```
C:\Users\HP\Desktop\22_sliding_puzzle
```

5. Click **Select Folder**. If prompted, click **Trust**.

You should see `main.py`, `game.py`, `puzzle.py` and `README.md` in the Explorer on the left.

Send me a screenshot once the folder is open. Next, we'll start fixing the first bug using your Vibe Coding tool. ✅

yes its there

Perfect! ✅ Your files are there. Now we'll fix the bugs **one by one**, without breaking your project.

Step 8 — Send me your original code

We'll use ChatGPT as your Vibe Coding tool so we can work with your actual code.

On your computer, go to this folder:

```
C:\Users\HP\Desktop\22_sliding_puzzle
```

Upload these **four files** into this chat using the **+ attachment button** beside the message box:

- `README.md`
- `main.py`
- `game.py`
- `puzzle.py`

How to upload them

1. Click the + button beside the ChatGPT message box.
2. Choose the option to attach files.
3. Open the folder shown above.

4. Select the four files and click **Open**.
5. Send them here.

Don't edit anything yet. I'll inspect your original code and help you fix Task 1 (making sure every puzzle is solvable), then we'll test it before moving to Task 2.

bro i cant see these files on my file explorer

BROO 😭 don't worry, your files are there! We already saw them in your VS Code terminal. We'll find them together.

Step 1 — Open Windows File Explorer

1. Press **Windows key + E** on your keyboard.
2. A File Explorer window will open.
3. Click the address bar at the top of that window.
4. Copy and paste this exact path:

C:\Users\HP\Desktop\22_sliding_puzzle

Copy folder path

5. Press **Enter**.

You should now see these files:

```
main.py
game.py
puzzle.py
README.md
requirements.txt
```

If you're trying to attach the files to this ChatGPT chat, use the same path in the file-selection window to navigate directly to the folder.

Send me a screenshot of what appears after you press Enter. We'll take the next step from there. ❤️

game.py Code	**main.py** Code	**puzzle.py** Code
	README(20261013... Document	**requirements(2026...** Document

BROO 👫 ❤️ **You uploaded all five files successfully!** I can read your original code, so we can fix the actual bugs instead of guessing.

We'll complete **Task 1 first**, test it, and then move to Tasks 2, 3 and 4.

Step 1 — Fix the first bug in `puzzle.py`

I found the issue in your code:

Python

```
random.shuffle(tiles)  # intentional solvability defect
```

Randomly shuffling the tiles can create an impossible puzzle. Your README asks you to start with the solved board and scramble it using legal moves. README.md

Do exactly this in VS Code

1. Open your `22_sliding_puzzle` folder.
2. Click `puzzle.py` in the left Explorer panel.
3. Select all the contents of `puzzle.py` and replace them with the corrected code below.
4. Press **Ctrl + S** to save.

Replace the contents of puzzle.py only. Leave main.py and game.py unchanged for now.

Python

```
import random


class Puzzle:
    def __init__(self, size=4):
        self.size = size
```

```
        self.board = self.make_board()

    def make_board(self):
        # Begin with the solved arrangement.
        tiles = list(range(1, self.size * self.size)) + [0]
        board = [
            tiles[r * self.size:(r + 1) * self.size]
            for r in range(self.size)
        ]

        # The blank starts at the bottom-right corner.
        blank_r = self.size - 1
        blank_c = self.size - 1
```

Step 2 — Test that the fix works

After saving `puzzle.py`, click the VS Code terminal at the bottom.

Run this command:

powershell

```
py -c "from puzzle import Puzzle; n=4; boards=[Puzzle(n).board for _ in range(100)]; assert all(sorted(x for r
```

Expected output:

Plain text

```
PASS: 100 valid, non-solved boards generated.
```

This verifies that 100 fresh boards each contain every tile exactly once and do not start in the solved arrangement. Because each board is constructed through legal moves from the solved state, each is solvable.

Step 3 — Run the game again

Next, run:

```powershell
py main.py
```

The game should display a scrambled puzzle as before. Try a few valid moves and then enter `q` to quit.

Stop here and send me a screenshot of your terminal after running the test command. Once Task 1 passes, we'll proceed to Task 2: completing the game when the puzzle is solved.



this came

BROOO YESSS 😭 ❤️ **Task 1 passed!** Your screenshot confirms this output:

```
PASS: 100 valid, non-solved boards generated.
```

Your game also starts successfully, so the fix is working. ✅

The `Can not find Conan command` notification on the bottom-right is unrelated to this Python test. You can ignore it for now.

Step 9 — Test Task 2: Completing the puzzle

I checked your original `game.py`. It already contains code that checks whether the puzzle is solved, prints `Solved!`, and exits the game. So let's **test it before changing anything unnecessarily**.

1. Exit the current game

Click inside the terminal at the bottom and type:

```
q
```

Press **Enter**.

2. Run this test command

Copy and paste this exact command into the terminal:

powershell

```
py -c "from game import SlidingPuzzle; g=SlidingPuzzle(); g.puzzle.size=3; g.puzzle.board=[[1,2,3],[4,5,6],[7,
```

This sets up a 3×3 puzzle that is exactly **one move away from being solved**.

3. Solve the puzzle

When the board appears, type:

Plain text

```
d
```

Press **Enter**.

The blank should move right, placing tile 8 in its correct position.

You should then see:

Plain text

```
1 2 3
4 5 6
7 8

Moves: 1
Solved!
```

The exact spacing of the output may differ.

Your task now: Run this test and send me a screenshot of the result. If it works, we'll mark Task 2 as tested and move to **Task 3 — adding 3×3, 4×4 and 5×5 size selection**.



BROOOO YESSS 🥳 ❤️ **Task 2 passed!**

Your screenshot shows the correct solved board, `Moves: 1`, and `Solved!`. The program returned to the PowerShell prompt, so the game ends correctly after completion.

Now let's implement **Task 3 — 3×3, 4×4 and 5×5 modes, with move counting and a timer**.

Step 10 — Update `game.py`

You only need to change `game.py` for this step. Your `main.py` can remain unchanged.

1. Open `game.py`

In the VS Code Explorer on the left, click `game.py`.

2. Replace the code

Select all the contents of `game.py` using **Ctrl + A**, and replace them with this code:

Python

```python
import time
from puzzle import Puzzle


class SlidingPuzzle:
    def __init__(self):
        self.size = self.choose_size()
        self.puzzle = Puzzle(self.size)

        # Session statistics are initialised once.
        self.moves = 0
        self.started = time.monotonic()

    def choose_size(self):
        while True:
            print("\nChoose Puzzle Size:")
            print("3 - 3x3")
            print("4 - 4x4")
            print("5 - 5x5")
```

3. Save the file

Press **Ctrl + S**.

Make sure `game.py` no longer has an unsaved-change indicator.

Step 11 — Test all three sizes

In the terminal at the bottom of VS Code, run:

powershell

```
py main.py
```

You should see:

Plain text

```
Choose Puzzle Size:
3 - 3x3
4 - 4x4
5 - 5x5
Enter 3, 4, or 5:
```

Test them one at a time.

- **Test A — 3×3**
  Enter `3` and press Enter. Confirm that the board has three rows and three columns.

- **Test B — 4×4**
  Quit using `q`, run `py main.py` again, and enter `4`. Confirm that the board has four rows and four columns.

- **Test C — 5×5**

Quit and restart again. Enter `5` and confirm that the board has five rows and five columns.

For each size, verify that the move count increases after a legal move and that the timer measures elapsed gameplay time. Invalid moves should not increase the move count.

Important: The timer and move count are stored in the game session, separately from the puzzle board. They are not reset simply because the board is displayed again.

Your next step

Run `py main.py` now and send me a screenshot showing the size-selection menu and the board after you enter `3`.

We'll verify Task 3 before moving on to the final feedback tests and your AFTER video.



BROOO YESSS 😭 ❤️ Your screenshot confirms that **Task 3 is working so far!**

- ✅ 3×3 mode launched successfully.
- ✅ 4×4 mode launched successfully.
- ✅ The move counter and timer are displayed.
- ✅ You can quit and restart the game.

Now we need to test 5×5 before moving on.

Step 12 — Test the 5×5 puzzle

Your 4×4 game is currently running in the terminal.

1. Quit the current game.

Click inside the terminal, type:

```
Plain text
q
```

Press Enter.

2. Start the game again.

```powershell
py main.py
```

3. Select 5×5.

When the menu appears, enter:

```
5
```

Press Enter.

You should see a board with **five rows and five columns**, with the move count and timer displayed below it.

Step 13 — Test invalid moves and move counting

While the 5×5 game is running:

1. Enter an invalid command such as `x`. It should be rejected, and the move count should remain at 0.
2. Enter a valid direction (`w`, `a`, `s` or `d`). If the move is legal, the move count should increase to 1.
3. Wait a few seconds and check that the timer increases.
4. Enter `q` to quit.

If a direction is impossible at the current blank position, the game should reject it without increasing the move count.

Send me a screenshot after you select 5. Then we'll finish testing Task 4, check that the movement instructions match the actual tile movement, and prepare your final AFTER video.



BROOO YESSS 😭 ❤️ **Your 5×5 puzzle is working too!**

Your screenshot confirms all three board sizes have launched successfully:

- ✅ 3×3
- ✅ 4×4
- ✅ 5×5
- ✅ Timer and move counter displayed

Now let's finish testing **Task 4 — valid and invalid move feedback**, as required by your README.

Step 14 — Test invalid commands

Your 5×5 game is still running.

1. Type `x` in the terminal and press Enter.

You should see:

```
Plain text

Invalid command. Use W/A/S/D or Q.
```

The move count should remain at `0`.

Step 15 — Test valid moves and impossible moves

Your current blank tile is in the fourth row. We'll move it to the top row to test an impossible move.

2. Type `w` **and press Enter three times**, one command at a time.

The move counter should increase by one for each successful move, reaching `3`.

3. Type `w` **one more time.**

Because the blank is now at the top edge, this move should be rejected:

```
Plain text

That move is not possible.
```

The move counter should remain at `3`. The board should not change.

4. Wait a few seconds. Confirm that the timer increases, then enter `q` to quit.

Step 16 — Record your AFTER video

Once those tests work, record your final video. You can show the 5×5 board, a valid move, a rejected move, the counter and timer.

Use Snipping Tool's recording feature again. Keep the video at least 10 seconds long and save it as `after.mp4`.

Do Steps 14 and 15 first and send me a screenshot. Then we'll check the final code and push your completed project to GitHub.

after.mp4.mp4
File

done taking video

Worked for 39s

BROOOO 😭 ❤️ **Your AFTER video is recorded!** I checked it: it's about **20 seconds long**, and it shows the 4×4 and 5×5 modes, the timer, valid moves, and an impossible move being rejected without increasing the move count. That covers the key behaviours we wanted to demonstrate.

Now let's push your corrected code to GitHub. You're nearly done!

Step 17 — Check your changed files

Go to the VS Code terminal at the bottom of your screen.

Type this command and press **Enter**:

powershell

```
git status
```

You should see the modified files, including `game.py` and `puzzle.py`.

Step 18 — Send me a screenshot

Please send me a screenshot of the `git status` output before running anything else. I'll check which files need to be committed so we don't accidentally upload unnecessary files.

One small detail: your uploaded video is named `after.mp4.mp4`. That's okay for now, but we'll make sure the final submission uses the clean filename `after.mp4`.

Next goal: push the fixed code to your personal GitHub fork, then finish preparing your `Lab-4` submission folder. ✅



Perfect! ❤️ Your screenshot confirms exactly what we need. Git detected the two modified files:

- `game.py` — updated game modes, timer and move counting.
- `puzzle.py` — fixed solvable starting boards.

There is also an untracked `__pycache__` folder. **We will not upload that folder.**

Step 18 — Stage only the corrected code

In your VS Code terminal, copy this command and press Enter:

```powershell
git add game.py puzzle.py
```

Now run:

```powershell
git status
```

You should see `game.py` and `puzzle.py` under **Changes to be committed**.

Step 19 — Commit your changes

Run this command:

```powershell
```

```
git commit -m "Fix sliding puzzle and add game modes"
```

Wait for Git to confirm that the commit was created.

Step 20 — Push to your personal GitHub repository

Run:

```powershell
git push origin main
```

Wait for the command to finish. It should upload your changes to your personal fork.

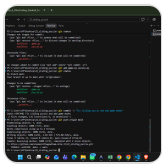
Step 21 — Verify on GitHub

Open your repository:

Open your 22_sliding_puzzle repository ↗

Refresh the page and check that the latest commit appears.

Do Steps 18–20 now, and send me a screenshot of the terminal after `git push origin main`. Then we'll finish organising your two videos and your complete AI chat history for submission.



Worked for 50s

BROOOO YESSS 🙌 🎉 **YOUR CODE HAS BEEN PUSHED SUCCESSFULLY TO GITHUB!**

Your screenshot confirms the push worked:

- ✅ `game.py` committed and uploaded.
- ✅ `puzzle.py` committed and uploaded.
- ✅ Your personal repository's `main` branch updated.
- ✅ `__pycache__` was not committed.

Your code is now on your personal fork: Open your Sliding Puzzle repository ↗.

Next: Prepare your final submission

We need to put your videos, updated code and chat history in your **course Lab repository**, inside a `Lab-4` folder. This is separate from your Sliding Puzzle repository.

Step 1 — Check your two videos

Make sure you have both recordings saved on your computer:

`before.mp4`	Original, unchanged game — at least 10 seconds.	Recorded
`after.mp4`	Fixed game with the new features — at least 10 seconds.	Recorded

Your uploaded after-video was named `after.mp4.mp4`. Rename the local file to `after.mp4` so the filename is clean.

Step 2 — Open your GitHub repositories

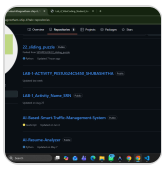
Click here:

Open your GitHub repositories ↗

Find the separate repository that you use for your course Lab submissions. Don't select `22_sliding_puzzle` again.

Send me a screenshot of your repository list. I'll help you identify the right Lab repository, then we'll create `Lab-4` and upload the required deliverables without missing anything.

Today 7:28 PM



Okay bro, I can see your repositories! ❤️ We're at the final submission stage now.

Step 22 — Open your Lab repository

On your screen, click this repository:

LAB-1-ACTIVITY_PES1UG24CS450_SHUBASHITHA

Your personalised repository containing your SRN.

Do not click `22_sliding_puzzle` — that is where your corrected game code is stored.

After opening the Lab repository

Send me a screenshot of the page you see. I'll guide you through creating the `Lab-4` folder and uploading your submission files one by one, including both videos and your complete AI chat history.

can i make a new repository cuz i dont wanna mix this with previous ones

BRO YES 😭❤️ You can create a **completely new GitHub repository just for Lab 4** so your previous lab submissions stay separate.

One thing to keep in mind: your handout says to upload the deliverables under `Lab-4` in your Lab repository. A separate repository keeps everything organised, but use it only if your instructor accepts a dedicated repository for this lab.

Step 1 — Create a new repository

Click this link:

Create a new GitHub repository

Separate from your previous labs and your puzzle-code repository.

Open GitHub's Create Repository page → ↗

Step 2 — Fill in the details

Use these settings:

Owner

shubashithagowtham-ship-it

Repository name

`LAB-4-VibeCoding-PES1UG24CS450` Copy

Description (optional)

Lab 4 — Vibe Coding: Sliding Puzzle, Scenario 22

Visibility

Public, unless your instructor requires private repositories. Public makes it easier for your instructor to view your submission.

Tick “Add a README file” so the repository starts with a file.

Then click **Create repository**.

Step 3 — Tell me when it's created

Once the new repository opens, send me a screenshot.

We'll then upload your deliverables into this structure:

```
LAB-4-VibeCoding-PES1UG24CS450/
└── Lab-4/
    ├── before.mp4
    ├── after.mp4
    ├── Updated-Code/
    │   ├── main.py
    │   ├── game.py
    │   ├── puzzle.py
    │   ├── README.md
    │   └── requirements.txt
    └── chat-history.pdf
```

Your corrected puzzle code is already pushed to your personal `22_sliding_puzzle` fork, so we won't disturb it. We'll keep this new repository dedicated to your **Lab 4 submission**. ✅

yes i created

YAYY BROO 😭 ❤️ Now we'll upload your Lab 4 submission to the new repository, keeping it separate from your previous labs.

Step 1 — Open your new repository

1. Go to GitHub ↗.

2. Click the new repository you just created: `LAB-4-VibeCoding-PES1UG24CS450`.
3. You should see its main page with your README file.

Step 2 — Create the `Lab-4` folder

On your new repository page:

1. Click **Add file**.
2. Select **Create new file**.
3. In the filename box at the top, enter:

```
Lab-4/README.md
```

4. In the editor, type:

```
Lab 4 – Vibe Coding: Scenario 22 Sliding Puzzle
```

5. Scroll down and click **Commit changes**.
6. Confirm the commit if GitHub asks.

This creates the `Lab-4` folder.

Step 3 — Create the code folder

Open the new `Lab-4` folder. Repeat the same procedure, but name the new file:

```
Updated-Code/README.md
```

For its contents, type:

```
Updated source code for Scenario 22 – Sliding Puzzle.
```

Click **Commit changes** again.

Your structure should now look like this:

LAB-4-VibeCoding-PES1UG24CS450

Lab-4/

README.md

Updated-Code/

Stop here and send me a screenshot of your new repository. Next we'll upload your five source files and both videos, then finish with your chat history.

no bro ill add to existing one only

BRO OKAY 😭 ❤️ We'll use your **existing Lab repository**. That actually matches your lab handout, which says to upload everything inside a `Lab-4` folder in your Lab repository.

Your corrected code is already pushed, so we won't touch that.

Step 1 — Open your existing Lab repository

Click here:

Open your existing Lab repository ↗

Make sure the repository name at the top is:

```
LAB-1-ACTIVITY_PES1UG24CS450_SHUBASHITHA
```

Don't open `22_sliding_puzzle` or the new repository you just created.

Step 2 — Create the `Lab-4` folder

Once the existing repository opens:

1. Click **Add file**.
2. Select **Create new file**.
3. In the filename field, type:

```
Lab-4/README.md
```

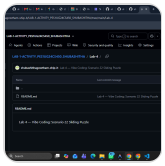
4. In the text area, type:

```
Lab 4 — Vibe Coding: Scenario 22 Sliding Puzzle
```

5. Click **Commit changes**.

This creates the `Lab-4` folder in your existing repository. Leave the new repository alone for now; we can decide later whether to delete it.

Send me a screenshot once you've opened your existing Lab repository. We'll upload the videos and code into `Lab-4` next, one step at a time.



BROO YESSS 😭 ❤️ You're in the **correct** `Lab-4` **folder** inside your existing Lab repository. We've successfully created the folder and its README.

Now we'll upload your corrected code.

Step 23 — Create the `Updated-Code` folder

On the GitHub page in your screenshot:

1. Click **Add file** near the top-right.
2. Select **Create new file**.
3. In the filename box, type exactly:

```
Updated-Code/README.md
```

4. In the text area, type:

```
Updated source code for Lab 4 — Scenario 22: Sliding Puzzle.
```

5. Scroll down and click **Commit changes**. Confirm the commit if prompted.

This creates the folder where we'll keep your corrected Python files.

Step 24 — Open the new folder

After committing, return to the `Lab-4` folder and click **Updated-Code**.

You should see its `README.md`.

Stop there and send me a screenshot. Next, I'll guide you through uploading all five project files from your computer into this folder, followed by your before and after videos.



Bro you're in the right place! ❤️ You're inside `Lab-4`, but I can see that the `Updated-Code` folder hasn't been created yet. Let's do that now.

Step 1 — Create the code folder

On the screen you sent:

1. Click **Add file** in the top-right corner.
2. Click **Create new file**.
3. In the filename box, type exactly:

```
Updated-Code/README.md
```

4. In the large text area, type:

```
Corrected source code for Lab 4 — Scenario 22: Sliding Puzzle.
```

5. Scroll down to **Commit changes**.
6. Click it and confirm the commit.

Step 2 — Check the folder

You should now see this structure:

Lab-4/

README.md

Updated-Code/

Send me another screenshot when you see `Updated-Code`. Then we'll upload `main.py`, `game.py`, `puzzle.py`, `README.md` and `requirements.txt` from your computer into that folder.



YES BROO ❤️ 😭 **Your** `Lab-4/Updated-Code` **folder has been created successfully!** You're in the right repository.

Now we'll upload your actual Python files from your computer.

Step 25 — Open `Updated-Code`

On your current screen:

1. Click the **Updated-Code** folder in the main panel.
2. You should see the `README.md` we created earlier.
3. We need to replace that placeholder README with your actual project README so the submission contains the original assignment instructions.

Send me a screenshot after you open `Updated-Code` **.** I'll show you exactly what to click next so we don't accidentally change anything in your previous lab folders.



BROO ❤️ you're in the correct folder! We can now upload your actual Python code. Your `Updated-Code` folder currently contains only the placeholder `README.md` , so let's add the files from your computer.

Step 26 — Upload your corrected code

1. Click `Add file` in the top-right corner.

2. Select `Upload files` **.**

3. Select your project files.

In the file-selection window, navigate to:

```
C:\Users\HP\Desktop\22_sliding_puzzle
```

Select these **four files**:

- **main.py**
- **game.py**
- **puzzle.py**
- **requirements.txt**

To select multiple files, hold **Ctrl** while clicking each filename, then click **Open**.

Do not select `__pycache__` **or the** `.git` **folder.** Also, leave `README.md` unselected for now because a placeholder with that filename already exists in the GitHub folder.

4. Commit the upload.

Once the four files appear on GitHub's upload page, scroll down, enter this commit message:

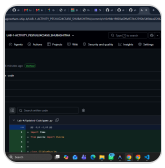
```
Add corrected Sliding Puzzle source code
```

Click **Commit changes**.

Step 27 — Verify the upload

You should now see `main.py`, `game.py`, `puzzle.py`, `requirements.txt` and the existing `README.md` inside `Lab-4/Updated-Code`.

Send me a screenshot after the upload is committed. Next, we'll replace the placeholder README with the actual project README, then upload your two videos and finish your chat-history deliverable.



BROOO YESSS 😭 ❤️ **Your corrected code has been uploaded successfully!**

Your screenshot confirms that all four files are committed inside `Lab-4/Updated-Code`:

- ✅ `main.py`
- ✅ `game.py`
- ✅ `puzzle.py`
- ✅ `requirements.txt`

Now we need to upload your two videos into the **main** `Lab-4` **folder**, not inside `Updated-Code`.

Step 28 — Upload your BEFORE and AFTER videos

1. Open the `Lab-4` folder

Click this link:

Open your Lab-4 folder on GitHub ↗

2. Start uploading

On that page:

1. Click **Add file**.
2. Select **Upload files**.
3. In the file-selection window, find your saved recordings.

Upload these two files:

`before.mp4`
Your original game recording.

`after.mp4`

Your fixed game recording.

Hold **Ctrl** while selecting both files, then click **Open**.

If your after-video is still named `after.mp4.mp4`, rename the local file to `after.mp4` first.

3. Commit the videos

Once both videos appear on GitHub's upload page, enter this commit message:

```
Add before and after gameplay videos
```

Click **Commit changes**.

Stop there and send me a screenshot. After the videos are uploaded, we'll finish the last deliverable: your complete AI chat history, so your Lab 4 submission is organised in one place. ✅



BROOOO ❤️ 😭 **Both videos are uploaded successfully!** Your screenshot confirms that `before.mp4` and `after.mp4` are inside your existing repository's `Lab-4` folder, alongside `Updated-Code`.

Your remaining deliverable is the **complete AI chat history**. Your handout asks for an exported document/PDF, and your project README also asks for the chat link, so let's prepare both.

Step 29 — Get your ChatGPT conversation link

1. Click the first Chrome tab at the top labelled **Sliding Puzzle Steps**.
2. Look near the top-right of the ChatGPT conversation for the **Share** button or share icon.
3. Click it and create a share link if prompted.
4. Click **Copy link**.

This link should open the conversation showing the prompts and coding assistance you used for the assignment.

Don't send me the link just yet. First, tell me whether you can see the Share button when you open that ChatGPT tab. I'll guide you through the next click, then we'll export the PDF and upload it into `Lab-4`.

1 selection

i didnt understand tell again

Bro don't worry 😭 ❤️ I'll explain slowly, one step at a time.

Step 29 — Get your ChatGPT conversation link

Step 1: Open ChatGPT

Look at the tabs at the very top of your Chrome window.

Click the **first tab on the left called "Sliding Puzzle Steps"**. That's your ChatGPT conversation.

Step 2: Find the Share button

Once ChatGPT opens:

1. Look at the **top-right corner of the ChatGPT page**.
2. Find the **Share** button or the share icon.
3. Click it.

Step 3: Copy the link

A small window should appear. Click **Create link** if shown, and then click **Copy link**.

⚠️ The link may allow anyone who has it to view the shared conversation, so share it only as required for your assignment.

For now, just do Step 1: click the "Sliding Puzzle Steps" tab and send me a screenshot. I'll show you exactly where to click next. ❤️

done

Good bro ❤️ Now let's do the next step!

Step 30 — Click Share in ChatGPT

Look at the **top-right corner of your ChatGPT conversation**.

1. Find the **Share** button or the icon that looks like an arrow pointing upward.
2. Click it.
3. A small window should open with an option such as **Create link**.

If you can't find the Share button, send me a screenshot of your ChatGPT page, and I'll point out exactly where it is.

For now, just click Share and tell me what appears.

i copied the link

YAYY BRO ❤️ 😭 You've got the ChatGPT conversation link! Now let's save it in your GitHub submission.

Step 31 — Upload your chat link to GitHub

1. Open your Lab-4 folder

Click this link:

Open your Lab-4 submission folder ↗

2. Create a new file

1. Click **Add file** in the top-right corner.
2. Click **Create new file**.
3. In the filename box, type:

```
chat-link.txt
```

4. Click inside the large text area and press **Ctrl + V** to paste the ChatGPT link you copied.

3. Save it

Scroll down to **Commit changes**.

Enter this commit message:

```
Add ChatGPT conversation link
```

Click **Commit changes**.

Now your instructor can find the conversation link in your Lab-4 folder.

Tell me once you've committed `chat-link.txt` . Then we'll create the required PDF of your chat history, which is the last deliverable. ❤️

ya done

BROOO ❤️ 😭 Almost done! Your `chat-link.txt` is uploaded. Now let's save a PDF of the conversation as well, so you have the chat history ready for submission.

Step 32 — Save your ChatGPT conversation as a PDF

1. Open your ChatGPT conversation

Click the **"Sliding Puzzle Steps"** tab at the top of Chrome.

2. Open the Print window

Press these keys together:

Ctrl + P

Print this conversation

A print window should appear.

3. Save as PDF

In the print window:

1. Find **Destination**.
2. Select **Save as PDF**.
3. Click **Save**.
4. Choose your Desktop as the location.
5. Name the file exactly:

```
chat-history.pdf
```

6. Click **Save** again.

Wait for the file to finish saving.

Step 33 — Upload the PDF to GitHub

Open your Lab-4 submission folder ↗.

Click **Add file** → **Upload files**. Select `chat-history.pdf` from your Desktop, then commit it with this message:

```
Add exported ChatGPT conversation
```

Important: Open the saved PDF and check that the conversation text is readable before uploading it.

Send me a screenshot when the PDF is uploaded. We'll do one final check of your submission folder against the assignment requirements. ✅